

Assignment 2: Solving Traveling Salesmen Problem using Local and Global Search

Introduction to AI, Instructor: Liaw Rung Tzuo | 廖容佐

Antony Wilson | 魏彥晨 (412262319)

May 06, 2025

1 Problem Definition

The assignment is to implement and test different search algorithms that solve the **traveling salesman problem (TSP)**. Given is a complete graph $G = (V, E, w)$ with vertex set V , edge set $E = V \times V$, and weight function $w : E \rightarrow R$. The algorithms implemented aim to find a tour on the graph with minimum cost:

$$\pi^* = \operatorname{argmin}_{\pi} \sum_{i=1}^n d(\pi_i, \pi_{i+1}) \quad (1)$$

Each vertex has an x and y coordinate, and the cost is defined as Euclidean distance. *Hill-climbing search*, *simulated annealing*, *genetic algorithm*, and improvements to these search algorithms will be implemented to solve the problem. The number of tours evaluated in a run is limited to $1000n$ for n cities.

2 Conceptual Approach

The search algorithms used for this assignment are at their core local searches (though extensions allow them to explore the global problem space). They do not start with an empty set and build upon it step by step to find an optimal solution. Rather, they begin with a complete solution and generate neighboring solutions that allow traversal of the solution space. For the traveling salesman problem, a set of vertices and their respective coordinates is given. A solution is an ordered set (implemented as a vector) of these vertices, containing each vertex once and only once. The cost of a solution is the sum of the Euclidean distances between every vertex and the following vertex within the set. Euclidean distance is defined as:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The graph in this assignment is complete, that is, each vertex has an edge with every other vertex. So, we can be sure that as long as each vertex appears

once within the set, it is a possible solution regardless of order. Valid solutions can therefore be created by randomly initializing the vector or arbitrarily swapping the vertices of an existing solution (*two-city swap*). A big increase in performance was achieved by using *2-Opt-Swap* instead of simple two-city swap.

3 Implementation

3.1 Hill-Climbing Search

The first implemented algorithm is straightforward since it traverses the solution space by generating neighbors of the current solution and choosing the neighbor with minimum cost as the next node. The most basic form of hill-climbing search, **steepest-descent hill climbing**, generates all possible neighbors and selects the best option. When all neighbors have a higher cost than the current node, the current node gets returned as the optimal solution. Although this gives a high chance of finding low-cost neighbors, the number of neighbors is $\binom{n}{2}$, which becomes incomputable quickly as n gets larger. The limit of $1000n$ solution checks given by the assignment is already reached with $n \geq 100$.

Another approach that can, therefore, be implemented is **first-choice hill climbing**. Rather than choosing the best option among all possible neighbors, random neighbors are generated. As soon as one with a cost that is lower than the current is found, it is set as the successor. This way there are only ever two nodes in memory, and the search can advance without having to exhaust all possibilities at every iteration. I set a limit for the maximum number of nodes generated per iteration. When that maximum is reached, the current node is accepted as the result and returned.

I also implemented **sideways moves** to combat encountering plateaus. However, in my tests, this showed little improvement in the result, probably due to two tours rarely being of equal (or almost equal) cost.

3.2 Simulated Annealing

The main issue in hill-climbing search is the possibility of the algorithm getting stuck in local minima. Simulated annealing aims to address this problem by accepting some tours that increase the cost. The probability of accepting bad solutions is high early on and decreases based on a temperature schedule with time. How unfavorable a solution is also affects the probability. The temperature schedule can have different forms; I implemented a typical exponential cooling:

$$T_{k+1} = \alpha T_k.$$

The acceptance probability is then defined by

$$P = e^{-\Delta E/T_k},$$

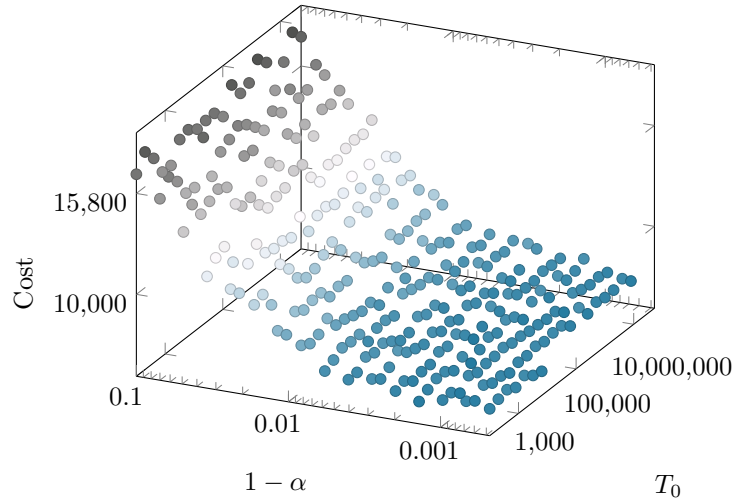


Figure 1: Optimal cost achieved with simulated annealing based on decay factor α and initial temperature T_0

with ΔE being the difference between the next possible solution and the current solution:

$$\Delta E = E_y - E_x.$$

As parameters for the algorithm, the initial temperature and the cooling factor can be adjusted for better results. As shown in Figure 1, my tests showed that changing the initial temperature had an inconsiderable effect. However, the cooling factor is clearly directly related to how good the result is. As it approaches zero, the cost for the final solution decreases. The downside is that increasing the cooling factor directly increases the number of nodes checked in the run. Due to the limit of $1000n$ node evaluations per run, simulated annealing may not show its full potential in these tests.

3.3 Genetic Algorithm

For the last algorithm, the way of generating new solutions is different. The genetic algorithm relies on two previous solutions being combined into one (or two) new ones, which then undergo mutation. For TSP there are several ways that this can be achieved. The important part is that when combining two tours, the program needs to make sure that every city still appears once and only once. It is not as straightforward as cutting to tours and stitching them together at a point. I implemented *edge recombination crossover (ERX)* for reproduction. Parameters that can be tweaked are the population size and how many generations are allowed to iterate.

Input	Implementation	Time (ms)	Tour Length
$n = 50$	Hill-climbing	37	8 076.93
	Simulated annealing	284	7 801.97
	Genetic algorithm	1 556	18 101.5
$n = 100$	Hill-climbing	131	16 222.5
	Simulated annealing	477	15 149.6
	Genetic algorithm	5 902	32 979.3
$n = 200$	Hill-climbing	409	108 702
	Simulated annealing	904	109 407
	Genetic algorithm	24 729	107 910
$n = 500$	Hill-climbing	1 907	10 049.2
	Simulated annealing	1 759	15 159.8
	Genetic algorithm	146 022	11 307.7
$n = 1000$	Hill-climbing	9 516	106 663
	Simulated annealing	3 748	244 789
	Genetic algorithm	90 223	106 463

Table 1: Measured performance of hill-climb search, simulated annealing, and genetic algorithm across different input sizes.

4 Conclusion

As can be seen from the test measurements in Table 1, improvements can still be made, especially for the genetic algorithm. Due to poorly optimized code or generally the higher computation needed for running and testing the genetic algorithm, I was unable to test and fine-tune parameters properly. Hill-climb search and simulated annealing appear to be optimized well.